Memory Management

→ Garbage collection
→ Reference counting
→ Best practices
→ GC module → Garbage collection
→ use generators ✓✓

(*) import sys

```
>>> sys.getrefcount(x)
```

→ returns no of pointers/references to the object that x points to.

eg >>> x = [ ]
print (sys.getrefcount(x)) → 2

```
>>> b = x
del x
print (sys.gc... (b)
```
→ 2

Garbage collection. → import gc

```
>>> gc.enable()
gc.disable()
gc.collect()
gc.get_stats()
gc.garbage
```

→ collects all garbage
→ gets stats
→ gets unreachable objects

Tracemalloc ? → consult your teacher.

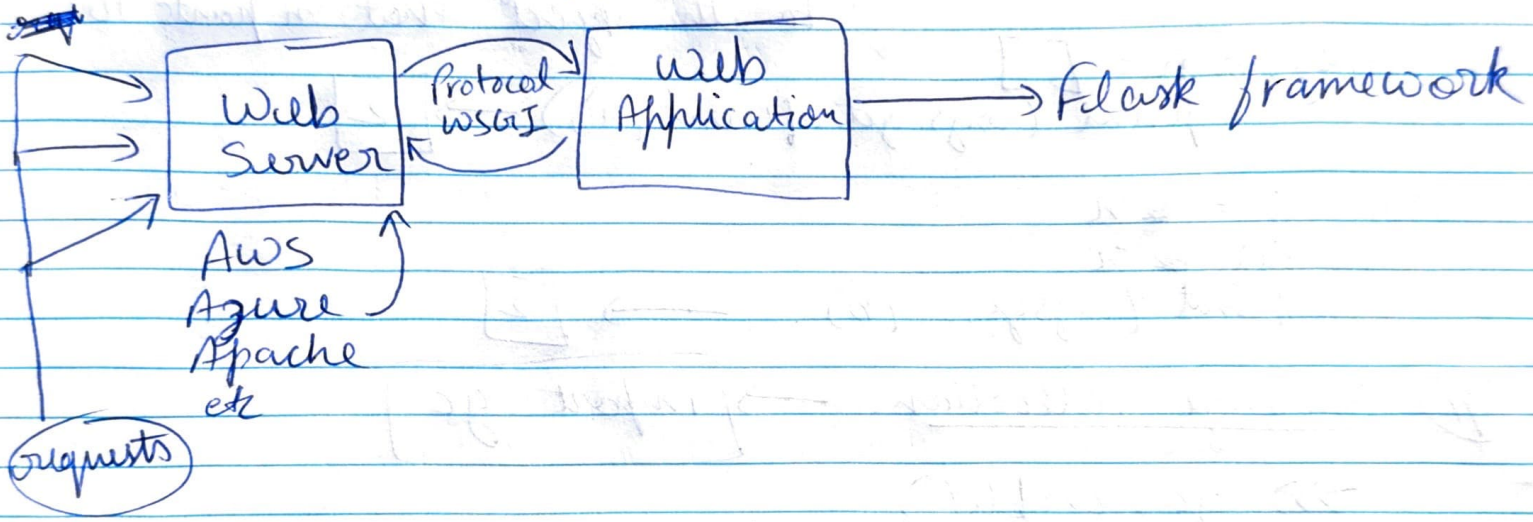
```
>>> import tracemalloc
```

Flask framework

① WSGI → Web Server Gateway Interface

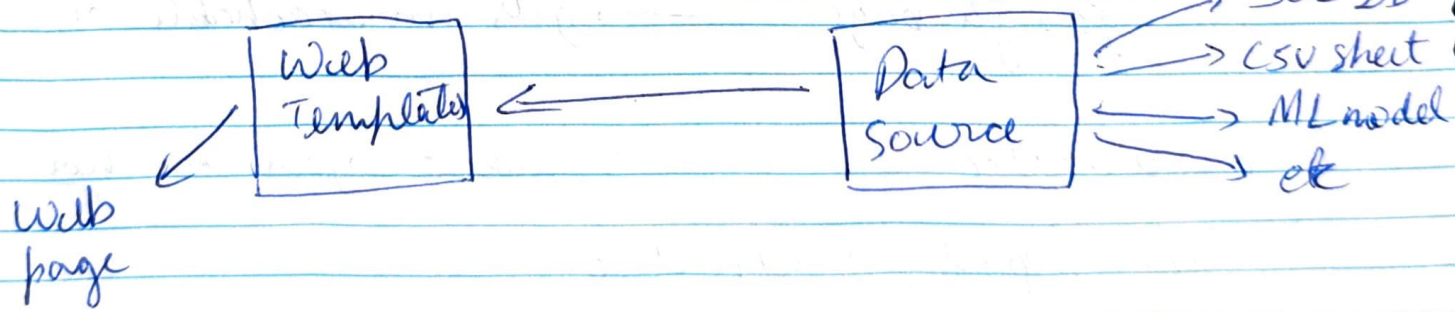
② Jinja2 → Template Engine

Flask → Web framework created in Python
→ used to develop end-to-end web application



Jinja 2 Template Engine

→ Web Template design
→ creates dynamic web pages



Jinja2 connects a web template to a data source

eg → login page to table data
→ image input page to ML model

pip install -r req.txt

Flask

~~>>> import~~

```
>>> from flask import Flask
```

app = Flask() → creating an object / instance of Flask class.

---

basic skeleton

```
>>> from flask import Flask
    app = Flask(__name__)
    if __name__ == '__main__':
        app.run()
```

entry point of flask app

VVV imp

Now,

```
>>> app = Flask(__name__)
    @app.route("/")
    def f1():
        return 'Welcome'

    @app.route("/index")
        return 'Yo Yo'

    if __name__ == '__main__':
        app.run(debug = True)
```

homepage function

for home directory of your app

for next directory

for development just same

server refreshes on its own

(*) app.run(debug = True)

→ You don't have to re-run the python file.
→ Just rename, & refresh the webpage.

(*) @app.route()

→ by default it points to home directory
→ good practise → @app.route("/")

(*) by default port → 5000
→ can be changed.

(*) Integration with html

```
@app.route('/index')
def index():
    return render_template("--.html")
```

new sub directory

for to render another template

(*) html document in the ~~source~~ template folder as code

→ searches in /templates

(*)(*) from flask import redirect_template

(A) from flask import ~~re~~redirect_template

→ sub-sub-directory ~~rede~~ & redirection

```
@app.route('/aa')
def aa():
    return 'Hey, this is a sub-dir'

@app.route('/aa/bb')
def bb()
    return redirect_template("bb.html")
```

→ ~~st~~ directory path → /aa/bb

root, 1-sub, 1-sub-sub

→ redirecting to 'bb.html'

→ templates folder
- > bb.html
- > .....
- > main.py

(A) → templates folder to be present in same directory as code.

→ ~~template~~ html files should be present in templates folder.

(A)(A) ✓

GET / POST methods

```
>>> from flask import Flask, render_template,
                                  request
>>> app = Flask(__name__)
>>> @app.route('/', methods=['GET','POST'])
def homepage():
    ~~if method.request == 'POST':~~
        ~~name =~~
    if request.method == 'POST':
        name = request.form['name']
        return f'Hello there {name}'
    return render_template('form.html')
```

homepage

by default, methods = ['GET'] only

if anything is posted/submitted by user, then this condition is TRUE.

request.method == ~~'POST'~~ 'GET' by default

if user posts any info,

request.method = 'POST'

form.html

```
< input ....
    name = 'name' >
```

name = request.form['name']

identifier

tells the interpreter to go look for the parameter name ~~pa~~ given in the html doc

The variable tag under which input is taken in html page form

Vimp

Action button in html / form method

→ To redirect to another template if any info is posted / submitted.

inside html doc

```
<!DOCTYPE html>
<html lang='en'>

  <form action='/submit' method='POST'>
    <.....>
    <input ....>

request.form['...']

  </form>
```

→ redirects to subdirectory /submit

if user inputs info, request.method == 'POST'

```
methods=['GET',
@app.route('/', ... 'POST'])
def homepage():
    if request.method == 'POST':
        .....
        .....
    return ....
```

```
@app.route('/submit', methods=['GET','POST'])
def submit():

    name = request.form['name']
    return f'Cherio {name}
```

→ if only methods = ['POST']

→ request.method always = 'POST'

This is necessary to retrieve name from form

if 'GET' is not there, can not access the directory directly

For security

⇒ If methods = ['POST'] only,

then only posting information can take you to the app route mentioned.

eg

```
@app.route('/submit', methods = ['POST'])
def submit():
    pass.
```

⇒ Can NOT access /submit directly.

⇒ Need to be redirected using action = '/submit'